

SRGGE - Lab Project

Santiago Escobar - June 2026

1 Missing functionalities

The following functionalities are not implemented in the project

1. Session 3: Vertex clustering using QEM
2. Session 7: Cell-to-cell visibility precomputation

2 Implemented functionalities

2.1 Session 1: File parsing

The application loads a scene from a text configuration file (map.txt). The file follows the structure presented in the course slides and allows the user to define both the museum floorplan and the models placed inside it.

The scene description is divided into two parts:

1. Floorplan definition.
2. Model definitions and instances.

Each model can have multiple instances. For every instance, the following transformations can be specified: Translation, Rotation, and Scale.

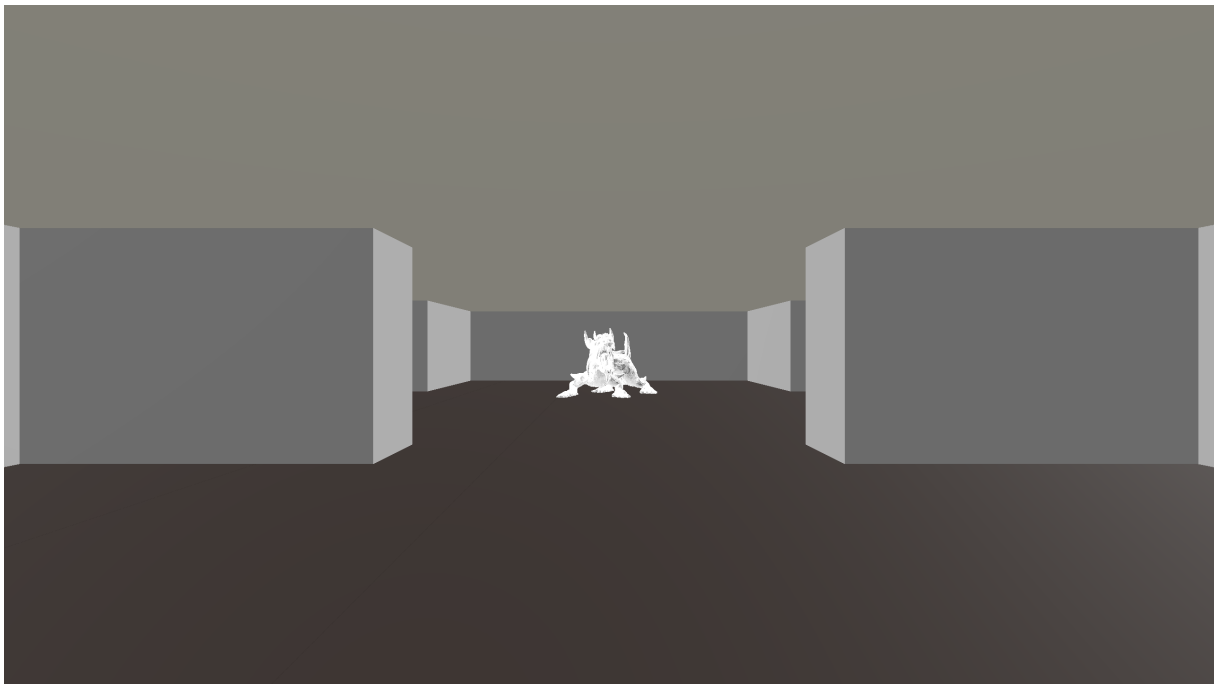


Figure 1: Floorplan and model instance loaded from map.txt.

2.2 Session 2: Vertex clustering

A vertex clustering simplification algorithm was implemented using a regular three-dimensional grid. For each grid cell, all vertices contained inside the cell are merged into a single representative vertex. The representative position is computed as the average of all vertices belonging to the cell.

Multiple Levels of Detail (LODs) are generated by varying the grid resolution. Larger cells produce more aggressive simplification and therefore lower geometric complexity. The generated LODs can be selected manually during execution using keys 0 through 4. By default, models are rendered using the coarsest available LOD.

LOD Caching

Generating simplified meshes can be computationally expensive. To avoid repeating this process on every execution, generated LODs are automatically stored on disk. When the application is launched again, previously generated LODs are loaded directly from the cache, significantly reducing startup time.

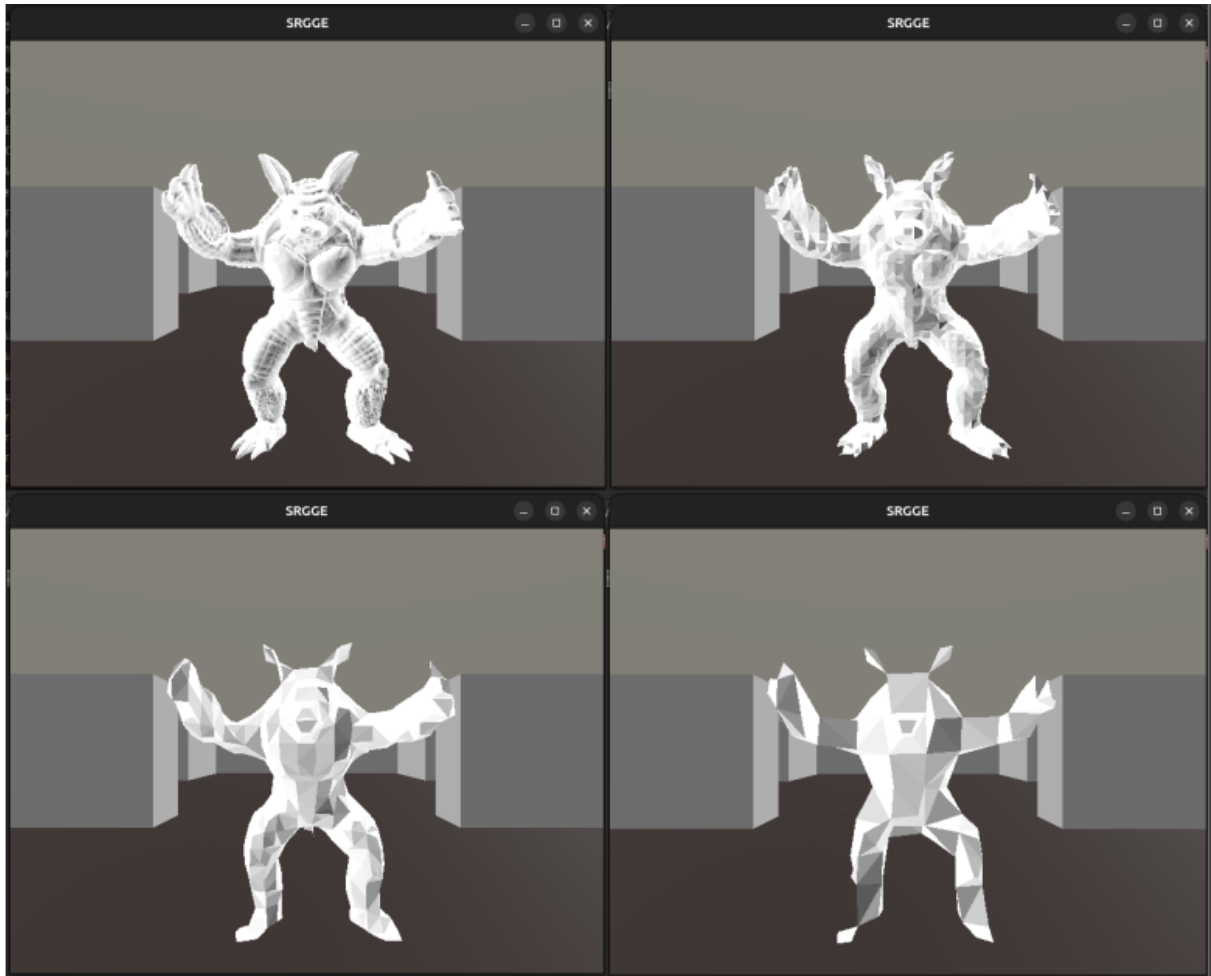


Figure 2: Comparison between different LOD levels using regular vertex clustering. Top-Left is the original model

2.3 Session 4: Thin feature preservation

To improve the preservation of thin geometric features, a normal clustering strategy was implemented on top of the regular vertex clustering algorithm. Instead of using a single representative per spatial cell, vertices are additionally separated according to their normal orientation. This

prevents unrelated surface regions from collapsing into the same representative vertex and helps preserve sharp edges and thin structures.

This implementation remains fully compatible with the LOD system and can be controlled using the same keys (0–4). The application can be launched using normal clustering through: `./SRGGE ../map.txt normalClustering`

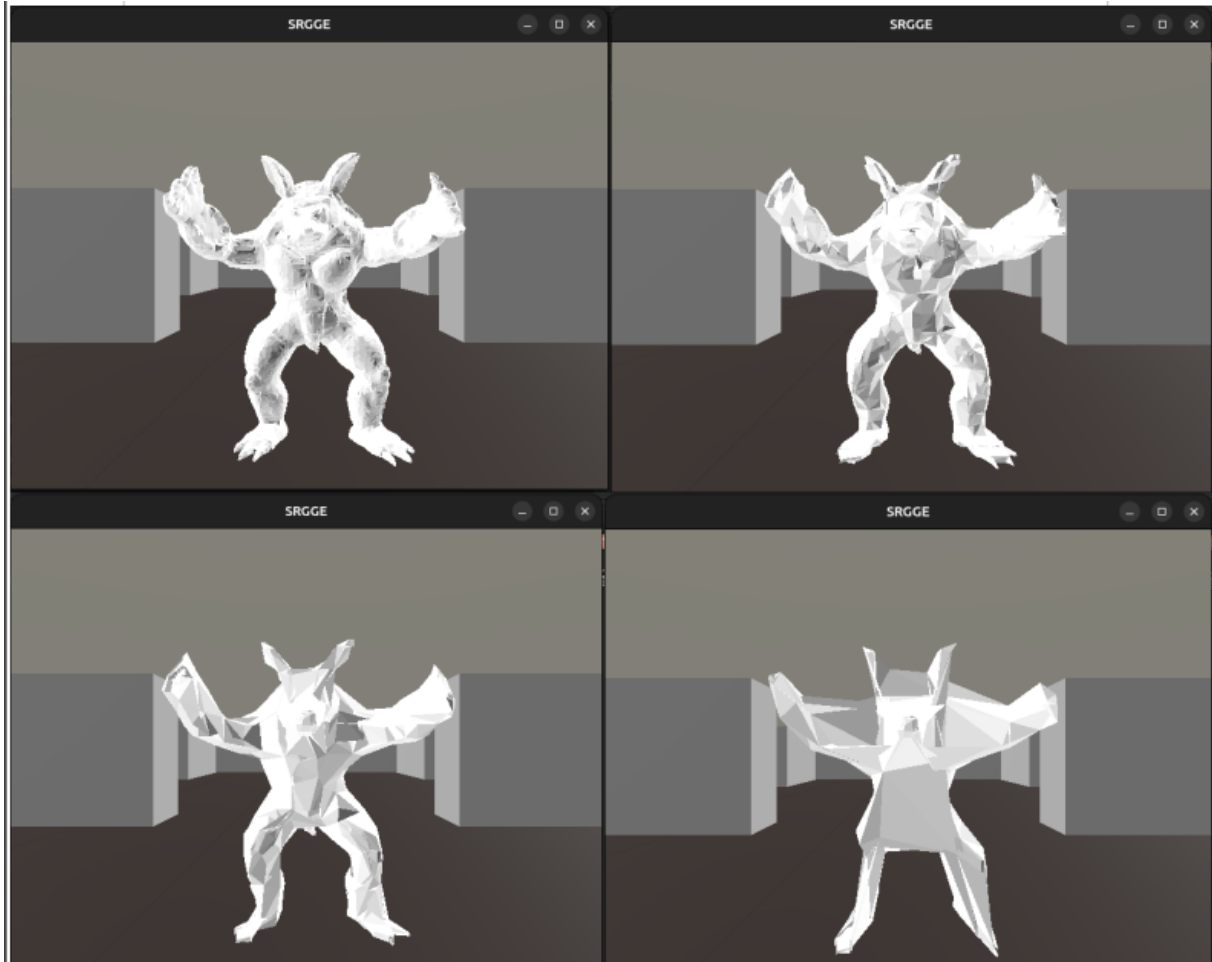


Figure 3: Generated LODs using normal clustering.

2.4 Session 5: Time critical rendering

A time-critical rendering system was implemented to dynamically select the level of detail of every object at runtime.

The system starts from the coarsest representation of all objects and progressively increases detail while respecting a global triangle budget. For each possible LOD upgrade, a benefit-to-cost ratio is computed. A greedy optimization algorithm repeatedly selects the upgrade that provides the highest benefit per additional triangle until the budget is exhausted.

This allows the renderer to maximize visual quality while maintaining a bounded rendering cost. The algorithm can be enabled during execution by pressing 5 and adjusted on runtime by pressing I or k

LOD Visualization

A debugging mode was implemented to visualize the selected LOD of every object. It uses a solid color for each LOD. This mode can be toggled by pressing C



Figure 4: Scene rendered using LOD visualization mode

2.5 Session 6: Hysteresis transition

To reduce visual popping caused by frequent LOD changes, hysteresis was added to the LOD selection process. Instead of immediately applying a newly selected LOD, the system requires the same decision to remain stable for a configurable number of consecutive frames before the transition is accepted. This prevents rapid oscillations when an object lies close to a decision boundary.

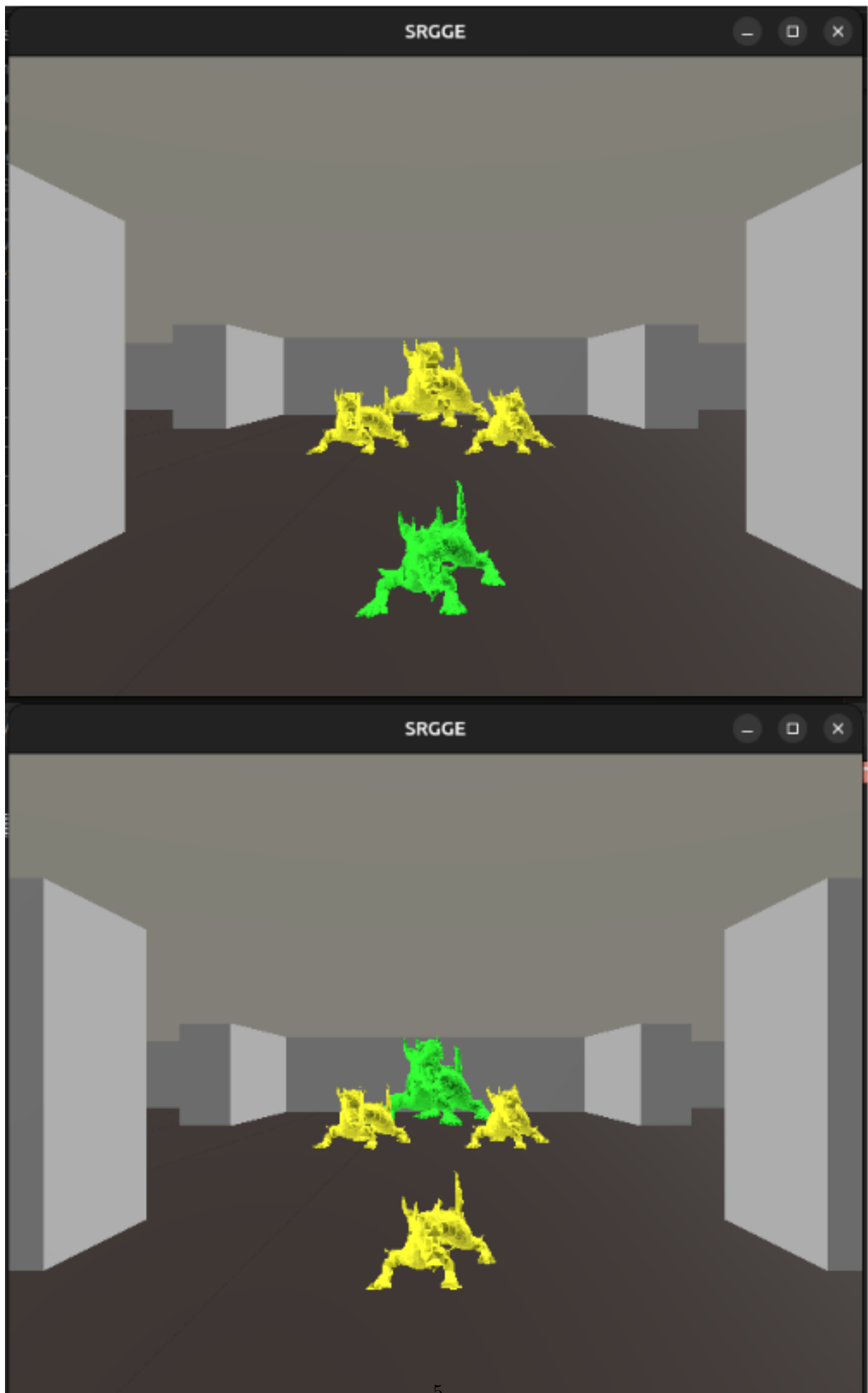


Figure 5: No serious popping when performing subtle changes in a short span of time